

Multi Pipeline ETL Framework for Web Server Log Analytics

Status Report + Architecture

Team Members

Ayush Mishra (IMT2023129)

Harsh Sinha (IMT2023571)

Syed Naveed Mohammed (IMT2023119)

Santhosh Vodnala (IMT2023622)

GitHub Repository

Multi Pipeline ETL Framework Project

April 28, 2026

Contents

1	Introduction	3
2	Proposed Architecture and Workflow	3
2.1	System Overview	3
2.2	End-to-End Data Flow	3
2.3	Pipeline Backend (MongoDB)	5
2.4	Design Principles	5
3	Parsing Strategy and Batching Approach	5
3.1	Log Format and Structure	5
3.2	Parsing Strategy	6
3.2.1	Regular Expression	6
3.2.2	Field Extraction and Transformation	7
3.3	Batching Approach	7
3.3.1	BatchReader — Streaming Lines from Disk	8
3.3.2	BatchProcessor — Parsing a Batch	8
3.3.3	Batch Size Configuration	9
3.3.4	Multi-File Processing	9
3.4	Handling Malformed Records	9
3.5	Pipeline Summary Output	10
4	Relational Reporting Database Schema	10
4.1	Schema Design	10
4.1.1	Query 1 Reporting Table	10
4.1.2	Query 2 Reporting Table	11
4.1.3	Query 3 Reporting Table	11
4.2	Metadata Columns (Common Across All Queries)	12
5	Query Outputs and Demonstrations	13
5.1	Storage in PostgreSQL	13
5.2	Query 1: Daily Traffic Summary	13
5.2.1	Implementation Details	13
5.2.2	Sample Output	13
5.2.3	Observations	14
5.3	Query 2: Top Requested Resources	14
5.3.1	Implementation Details	14
5.3.2	Sample Output	15

5.3.3	Observations	16
5.4	Query 3: Hourly Error Analysis	16
5.4.1	Implementation Details	16
5.4.2	Sample Output	17
5.4.3	Observations	17
6	Runtime Module	18
	Conclusion	18

1. Introduction

This report presents the implementation status and architecture of the MongoDB-based ETL pipeline for web server log analysis. It highlights the current progress, system design, and workflow used to process and aggregate log data using MongoDB, along with storage of results in PostgreSQL.

2. Proposed Architecture and Workflow

2.1 System Overview

The system implements a MongoDB-based ETL pipeline for processing large-scale web server logs. It focuses on transforming raw log data into structured format, processing it in batches, and performing efficient aggregation using MongoDB.

The architecture is modular, separating parsing, batching, storage, and query execution. This design enables scalability and simplifies extension to other pipelines in future phases.

2.2 End-to-End Data Flow

Figure 1 illustrates the overall workflow of the system.

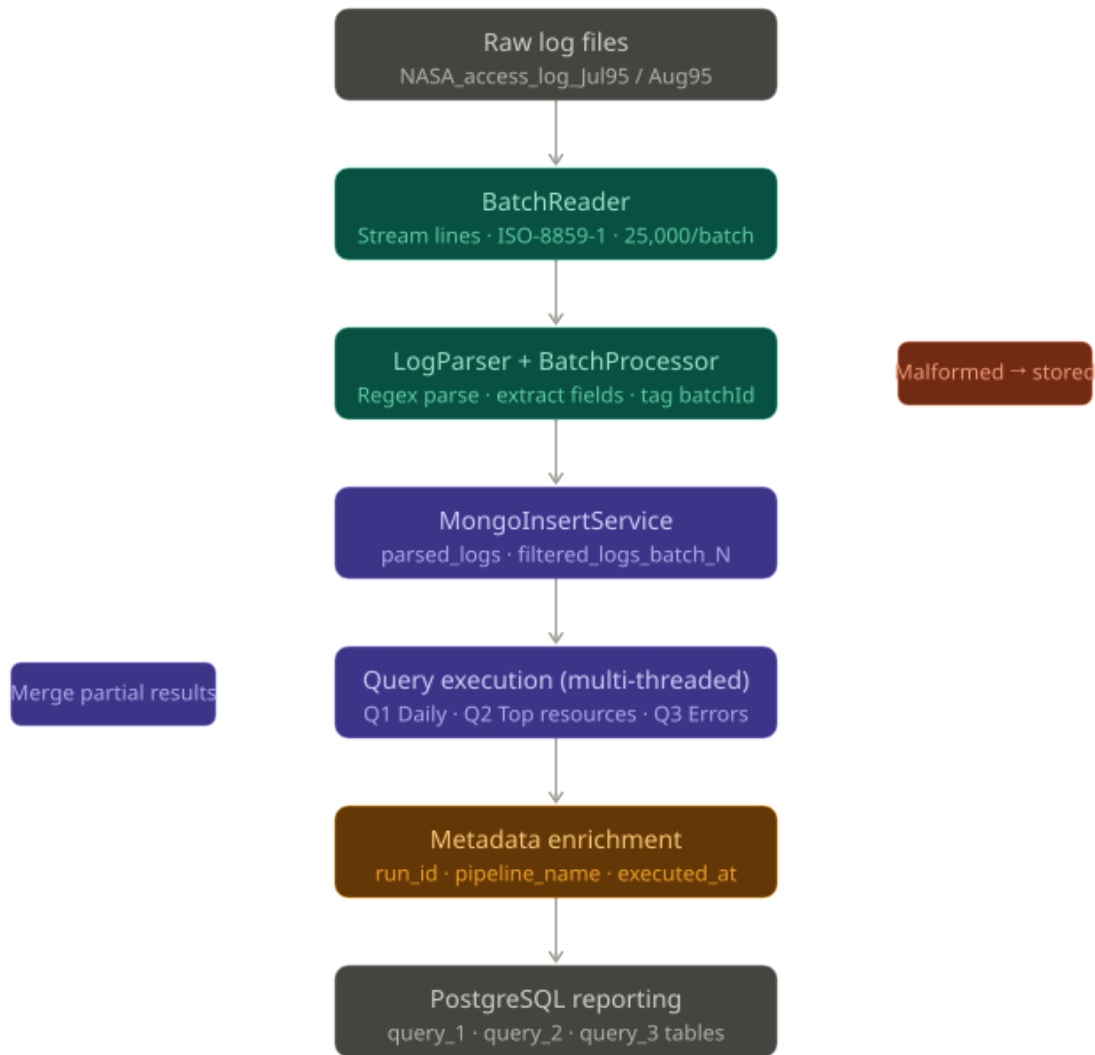


Figure 1: MongoDB ETL Pipeline Architecture

The data flows through the following stages:

- **Input Layer:** Raw NASA log files are read from disk.
- **Parsing Layer:** Logs are converted into structured records containing fields such as date, status code, and bytes.
- **Batching Layer:** Records are grouped into batches to enable scalable and parallel processing.
- **MongoDB Storage:** Each batch is stored in a separate MongoDB collection.
- **Processing Layer:** Aggregation queries are executed on each batch collection using MongoDB pipelines.

- **Global Aggregation:** Partial results from all batches are merged to produce final aggregated results.
- **Output Layer:** Final results are enriched with metadata and stored in PostgreSQL for reporting.

2.3 Pipeline Backend (MongoDB)

The MongoDB pipeline serves as the primary execution backend in this phase. It uses aggregation pipelines to compute query results efficiently.

Batch-level processing is executed in parallel using multi-threading, allowing multiple collections to be processed simultaneously. This improves performance and simulates distributed data processing.

The aggregation results are then combined to produce a global output, ensuring correctness across all batches.

2.4 Design Principles

The system is designed based on the following principles:

- **Modularity:** Parsing, batching, and query execution are implemented as independent components.
- **Scalability:** Batch-based processing and parallel execution enable handling of large datasets.
- **Extensibility:** The architecture can be extended to support additional pipelines in future phases.
- **Consistency:** Aggregation logic ensures accurate results across all batches.

3. Parsing Strategy and Batching Approach

This section describes how raw NASA HTTP server log files are read, parsed into structured records, grouped into batches for processing, and how malformed entries are handled throughout the pipeline. The implementation is spread across four classes in the `common` module: `LogParser`, `BatchProcessor`, `BatchReader`, and `BatchResult`.

3.1 Log Format and Structure

The input dataset consists of two raw NASA HTTP server access log files: `NASA_access_log_Jul95` and `NASA_access_log_Aug95`, together totalling approximately 373 MB of uncompressed log data.

Each line in the log follows the Combined Log Format (CLF), a standard format used by Apache and NCSA HTTP servers. A representative log line is shown below:

Listing 1: Sample NASA HTTP Log Entry

```
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/
apollo/ HTTP/1.0" 200 6245
```

The fields present in each log entry are as follows:

Table 1: NASA HTTP Log Fields

Field	Example Value	Description
Host	199.72.81.55	Client hostname or IP address
Ident	-	RFC 1413 identity (always -)
Auth User	-	Authenticated username (always -)
Timestamp	01/Jul/1995:00:00:01 -0400	Request date and time with timezone
Request	GET /history/apollo/ HTTP/1.0	Full HTTP request line
Status Code	200	HTTP response status code
Bytes	6245	Response size in bytes (- if unknown)

3.2 Parsing Strategy

Parsing is handled entirely by the `LogParser` class (`common/src/main/java/com/example/util/LogParser`). The parser applies a single compiled regular expression to each raw log line and extracts all relevant fields in one pass.

3.2.1 Regular Expression

The core pattern used is:

Listing 2: Log Parsing Regex Pattern

```
^(\S+) \S+ \S+ \[(.*?)\] "(.*)" (\d{3}) (\S+)$
```

The five capture groups in this pattern are described in the table below:

Table 2: Regex Capture Groups and Their Meanings

Group	Pattern	Field	Description
1	(\S+)	Host	Non-whitespace token — client IP or hostname
2	(.*?)	Raw Timestamp	Content inside [] — full date-time string
3	(.*?)	Request	Content inside " " — HTTP method, path, protocol
4	(\d{3})	Status Code	Exactly 3 digits — HTTP response code
5	(\S+)	Bytes	Non-whitespace — response size or -

3.2.2 Field Extraction and Transformation

Once the regex matches, the following transformations are applied before populating a `ParsedLog` object:

- **Timestamp parsing:** The raw timestamp (e.g. 01/Jul/1995:00:00:01 -0400) is split on ":" to separate the date part from the hour. The date is parsed using `DateTimeFormatter` with the pattern `dd/MMM/yyyy` and then re-formatted to ISO-8601 (`yyyy-MM-dd`) for downstream storage. The hour is extracted as an integer directly.
- **Request line parsing:** The request string (e.g. GET /history/apollo/ HTTP/1.0) is split on whitespace into method, path, and protocol. If fewer than three tokens are found, the record is flagged as malformed.
- **Bytes field:** If the bytes field equals "-", it is stored as 0. Otherwise it is parsed as a long.
- **Batch ID injection:** Each `ParsedLog` record is stamped with the `batchId` of the batch it belongs to, enabling per-batch filtering and aggregation downstream.

The resulting `ParsedLog` model captures the following fields:

Listing 3: `ParsedLog` Model Fields

```
1 private String host;  
2 private String rawTimestamp;  
3 private String date;           // ISO-8601 format: yyyy-MM-dd  
4 private int    hour;           // 0-23  
5 private String method;         // GET, POST, HEAD, etc.  
6 private String path;           // Requested resource path  
7 private String protocol;       // HTTP/1.0 or HTTP/1.1  
8 private int    status;         // HTTP status code  
9 private long   bytes;          // Response size (0 if unknown)  
10 private boolean malformed;     // true if parsing failed  
11 private int    batchId;        // Batch this record belongs to
```

3.3 Batching Approach

Large log files are not loaded entirely into memory. Instead, they are streamed and consumed in fixed-size batches. This design keeps memory usage bounded and naturally parallelises with the multi-threaded query execution layer.

3.3.1 BatchReader — Streaming Lines from Disk

The `BatchReader` class wraps a `BufferedReader` opened with ISO-8859-1 encoding (required because the NASA logs contain non-UTF-8 byte sequences). It exposes a single method, `readNextBatch(int batchSize)`, which reads up to `batchSize` lines from the file and returns them as a `List<String>`. An empty list signals end-of-file. `BatchReader` implements `AutoCloseable` so it is safely used inside a `try-with-resources` block.

Listing 4: BatchReader Core Method

```
1 public List<String> readNextBatch(int batchSize) throws
   IOException {
2     List<String> lines = new ArrayList<>(batchSize);
3     String line;
4     while (lines.size() < batchSize &&
5           (line = reader.readLine()) != null) {
6         lines.add(line);
7     }
8     return lines;
9 }
```

3.3.2 BatchProcessor — Parsing a Batch

The `BatchProcessor` class receives a list of raw lines and a `batchId`, calls `LogParser.parse()` on every line, counts valid and malformed records, and returns a `BatchResult` containing all `ParsedLog` objects plus the summary counters.

Listing 5: BatchProcessor Core Method

```
1 public static BatchResult processBatch(
2     List<String> rawLines, int batchId) {
3
4     List<ParsedLog> parsedLogs = new ArrayList<>(rawLines.size())
5     ;
6     int malformedCount = 0;
7
8     for (String line : rawLines) {
9         ParsedLog log = LogParser.parse(line, batchId);
10        if (log.isMalformed()) malformedCount++;
11        parsedLogs.add(log);
12    }
13
14    return BatchResult.builder()
15        .batchId(batchId)
```

```
15         .totalRecords(rawLines.size())
16         .malformedRecords(malformedCount)
17         .validRecords(rawLines.size() - malformedCount)
18         .parsedLogs(parsedLogs)
19         .build();
20     }
```

3.3.3 Batch Size Configuration

The batch size is externalised in `app.properties` and read at startup via `ConfigReader`:

Listing 6: `app.properties` Configuration

```
1 batch.size=25000
2 input.file.paths=./data/NASA_access_log_Jul95,
3                 ./data/NASA_access_log_Aug95
4 mongo.clear.before.run=true
```

With a batch size of 25 000 lines and approximately 3.3 million total log entries across both files, the pipeline produces around 133 batches in a full run. Each batch is assigned a monotonically incrementing integer `batchId` starting from 1, which is carried through every downstream stage.

3.3.4 Multi-File Processing

The pipeline iterates over multiple input file paths supplied as a comma-separated list. For each file a fresh `BatchReader` is opened and batches are consumed until EOF, after which the reader is closed and the next file begins. The `batchId` counter is *not* reset between files, ensuring globally unique batch identifiers across the entire run.

3.4 Handling Malformed Records

A record is classified as malformed under any of the following conditions:

- The raw log line does not match the regex pattern (missing fields, unexpected delimiters, truncated lines).
- The request field contains fewer than three whitespace-separated tokens, making method/path/protocol extraction impossible.
- Any numeric field (`status`, `bytes`, `hour`) throws a `NumberFormatException` during parsing.
- Any other unexpected exception is raised inside the `parse()` method.

Malformed records are *not* discarded. They are included in the `BatchResult` with `malformed = true` and are stored in the `parsed_logs` MongoDB collection alongside valid records. Only valid records (`malformed = false`) are written to `filtered_logs` and the per-batch `filtered_logs_batch_N` collections used for aggregation queries.

Per-batch malformed counts are persisted to the `batch_metadata` MongoDB collection. A global malformed count is accumulated across all batches and included in the final pipeline summary stored in `pipeline_run_metadata`.

3.5 Pipeline Summary Output

At the end of a full pipeline run, `MongoPipelineMain` prints a summary to stdout and persists the same data to MongoDB. The summary includes total records processed, valid and malformed counts, number of batches, average batch size, and total wall-clock execution time.

Table 3: Pipeline Summary — Full NASA Dataset Run

Metric	Value
Total Records Processed	3,461,613
Total Valid Records	3,461,612
Total Malformed Records	1
Total Batches	139
Average Batch Size	24,903.69
Total Execution Time (ms)	40,487

4. Relational Reporting Database Schema

4.1 Schema Design

4.1.1 Query 1 Reporting Table

The final output of Query 1 is stored in PostgreSQL table `public.query_1`. This table stores daily traffic summaries aggregated by log date and status code, along with execution metadata.

Table 4: Schema of PostgreSQL Table `public.query_1`

Column	Type	Purpose
id	SERIAL	Primary key
log_date	DATE	Log date of requests
status_code	INTEGER	HTTP response status code
request_count	BIGINT	Total number of requests
total_bytes	BIGINT	Total bytes transferred
batch_id	TEXT	Contributing batch IDs
run_id	TEXT	Unique execution identifier
pipeline_name	TEXT	Backend used for execution
executed_at	TIMESTAMP	Execution timestamp

4.1.2 Query 2 Reporting Table

The final output of Query 2 is stored in PostgreSQL table `public.query_2`. The table stores both aggregated analytics results and execution metadata so that repeated runs from different pipelines can be compared later.

Table 5: Schema of PostgreSQL Table `public.query_2`

Column	Type	Purpose
id	SERIAL	Primary key
resource_path	TEXT	Requested resource path
request_count	INTEGER	Total number of requests
total_bytes	BIGINT	Total bytes transferred
distinct_hosts	INTEGER	Number of unique requesting hosts
batch_id	TEXT	Contributing batch IDs
run_id	TEXT	Unique execution identifier
pipeline_name	TEXT	Backend used for execution
executed_at	TIMESTAMP	Execution timestamp

4.1.3 Query 3 Reporting Table

The final output of Query 3 is stored in PostgreSQL table `public.query_3`. This table stores hourly error analysis, including error counts, total requests, and computed error rates, along with execution metadata.

Table 6: Schema of PostgreSQL Table `public.query_3`

Column	Type	Purpose
log_date	TEXT	Log date of requests
log_hour	INTEGER	Hour of the request
error_request_count	INTEGER	Number of error requests (4xx/5xx)
total_request_count	INTEGER	Total number of requests
error_rate	DOUBLE PRECISION	Percentage of error requests
distinct_error_hosts	INTEGER	Unique hosts causing errors
batch_id	TEXT	Contributing batch IDs
run_id	TEXT	Unique execution identifier
pipeline_name	TEXT	Backend used for execution
executed_at	TIMESTAMP	Execution timestamp

4.2 Metadata Columns (Common Across All Queries)

The system stores additional metadata along with each aggregated result row in PostgreSQL. These metadata attributes are common across all queries (Query 1, Query 2, and Query 3) and help in tracking execution details, debugging, and comparing results across different pipelines.

- **pipeline_name:** Identifies the execution backend used to compute the result (e.g., MongoDB).
- **run_id:** A unique identifier (UUID) generated once per query execution. This allows grouping all rows belonging to the same run.
- **batch_id:** Indicates the set of batches that contributed to a particular aggregated result. For example, 1+2+3+4 represents aggregation across multiple batches.
- **executed_at:** Timestamp representing when the results were inserted into PostgreSQL. This is useful for tracking execution time and maintaining historical records.

These metadata fields ensure consistency across all query outputs and enable reproducibility, performance analysis, and cross-pipeline comparisons.

5. Query Outputs and Demonstrations

5.1 Storage in PostgreSQL

Instead of returning results only to the console, the system directly inserts the aggregated output into PostgreSQL using a shared insertion service. This ensures persistence and enables further querying and reporting. Further, for identifying the pipeline run, we store the results of the queries in a database in postgresql named "mongodb".

5.2 Query 1: Daily Traffic Summary

Query 1 computes the daily traffic summary by aggregating web server logs based on **log date** and **status code**. For each combination, the system calculates:

- Total number of requests
- Total number of bytes transferred

5.2.1 Implementation Details

The query is executed using a multi-stage approach:

- Each batch of logs is stored in a separate MongoDB collection.
- Aggregation is performed independently on each batch using MongoDB's `$group` operator.
- Partial results from all batches are merged to compute global aggregates.
- Multi-threading is used to process batch collections in parallel, improving performance.
- The final results are sorted by log date and status code.

5.2.2 Sample Output

Figure 2 shows a sample of the Query 1 output stored in PostgreSQL.

id	log_date	status_code	request_count	total_bytes	batch_id	run_id	pipeline_name	executed_at
2	1995-07-01	200	58033	1617409574	1+2+3	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
3	1995-07-01	302	2568	218465	1+2+3	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
4	1995-07-01	304	3797	0	1+2+3	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
5	1995-07-01	404	316	0	1+2+3	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
6	1995-07-02	200	54492	1531584293	3+4+5	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
7	1995-07-02	302	2378	195795	3+4+5	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
8	1995-07-02	304	3107	0	3+4+5	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
9	1995-07-02	403	5	0	3+4+5	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
10	1995-07-02	404	291	0	3+4+5	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
11	1995-07-03	200	81351	2085066566	5+6+7+8+9	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
12	1995-07-03	302	3122	261797	5+6+7+8+9	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
13	1995-07-03	304	4582	0	5+6+7+8+9	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
14	1995-07-03	403	2	0	6+8	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
15	1995-07-03	404	474	0	6+7+8+9	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
16	1995-07-03	500	53	0	7	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
17	1995-07-04	200	63069	1744266570	9+10+11+12	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
18	1995-07-04	302	2513	206813	9+10+11+12	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
19	1995-07-04	304	4507	0	9+10+11+12	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
20	1995-07-04	403	4	0	11	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
21	1995-07-04	404	359	0	9+10+11+12	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
22	1995-07-05	200	85080	2050959436	12+13+14+15+16	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
23	1995-07-05	302	3349	282418	12+13+14+15+16	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
24	1995-07-05	304	5646	0	12+13+14+15+16	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
25	1995-07-05	403	2	0	13+14	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
26	1995-07-05	404	497	0	12+13+14+15+16	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
27	1995-07-05	501	1	0	14	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
28	1995-07-06	200	91643	2218747557	16+17+18+19+20	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
29	1995-07-06	302	3197	264274	16+17+18+19+20	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
30	1995-07-06	304	5475	0	16+17+18+19+20	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129
31	1995-07-06	403	5	0	17+18+19	1c47554f-f752-4a86-845b-4c61289cb535	mongodb	2026-04-28 02:35:28.68129

Figure 2: Query 1: Daily Traffic Summary Output

5.2.3 Observations

- Status code 200 dominates the traffic, indicating successful requests.
- Some status codes such as 304 and 404 have zero byte transfers, which is expected for cached or failed requests.
- Batch identifiers confirm that multiple batches contribute to the final aggregated results.
- The aggregation correctly combines results across all batches, validating the global merge logic.

5.3 Query 2: Top Requested Resources

Query 2 computes the top 20 requested resource paths by aggregating web server logs based on **resource path**. For each selected resource, the system calculates:

- Total number of requests
- Total number of bytes transferred
- Number of distinct hosts requesting that resource

5.3.1 Implementation Details

The query is executed using a multi-stage approach:

- Each batch of logs is stored in a separate MongoDB collection.
- Aggregation is performed independently on each batch using MongoDB's `$group` operator grouped by `path`.

- Each batch computes local request count, total bytes, and distinct hosts.
- Partial results from all batches are merged in Java to compute global aggregates.
- Multi-threading is used to process batch collections in parallel, improving performance.
- The global top 20 resources are selected using request count ranking.
- The final output is displayed in ascending order of request count for readability.

5.3.2 Sample Output

The console output of query 2 is shown below. Following that is a screenshot from pgadmin showing the data in Postgresql database.

```

Flushed + Inserted 20 rows into query_2
resource_path | request_count | total_bytes | distinct_hosts | batches | run_id | pipeline | executed_at
-----
/history/apollo/images/footprint-logo.gif | 26287 | 100157364 | 17126 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/icons/blank.xbm | 28827 | 13201424 | 17838 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/shuttle/missions/sts-69/sts-69-patch-small.gif | 29101 | 212028657 | 18061 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/icons/menu.xbm | 29166 | 13896463 | 17811 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/shuttle/countdown/liftoff.html | 29839 | 111958733 | 14550 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/shuttle/missions/sts-69/mission-sts-69.html | 31554 | 317116549 | 18161 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/hb/cdt_main.pl | 39839 | 143854352 | 12643 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/images/launchmedium.gif | 40651 | 445305357 | 24823 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/shuttle/missions/missions.html | 47263 | 379051096 | 27537 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/images/ksclogosmall.gif | 61344 | 201833375 | 38812 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/shuttle/countdown/ | 64696 | 254973856 | 36878 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/history/apollo/images/apollo-logo1.gif | 68858 | 71737161 | 38380 | 28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/images/launch-logo.gif | 75960 | 114606552 | 42524 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/ksc.html | 83691 | 562133282 | 29146 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/images/ksclogo-medium.gif | 121286 | 628025692 | 49419 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/images/WORLD-logosmall.gif | 125676 | 73624119 | 53985 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/images/USA-logosmall.gif | 126820 | 25932114 | 54429 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/images/MOSAIC-logosmall.gif | 127656 | 40500273 | 54836 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/images/KSC-logosmall.gif | 164813 | 169168020 | 85100 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
/images/NASA-logosmall.gif | 208437 | 131494656 | 81432 | 1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56 | 1 | 1 | 2024-09-10 10:10:10.123
Process finished with exit code 0

```

Figure 3: Query 2: Top Requested Resources Console Output

Query	Query History	Scratch Pad				
1	SELECT * FROM public.query_2					
Data Output Messages Notifications						
Showing rows: 1 to 20						
Page No: 1 of 1						
	Id [PK] integer	resource_path text	request_count integer	total_bytes bigint	distinct_hosts integer	batch_id text
1		/history/apollo/images/footprint-logo.gif	26287	100157364	17126	1+2+3+4+5+6+7+8+9+10+11+12+13+14+15+16+17+18+19+20+21+22+23+24+25+26+27+28+29+30+31+32+33+34+35+36+37+38+39+40+41+42+43+44+45+46+47+48+49+50+51+52+53+54+55+56+57+58+59+60+61+62+63+64+65+66+67+68+69+70+71+72+73+74+75+76+77+78+79+80+81+82+83+84+85+86+87+88+89+90+91+92+93+94+95+96+97+98+99+100+101+102+103+104+105+106+107+108+109+110+111+112+113+114+115+116+117+118+119+120+121+122+123+124+125+126+127+128+129+130+131+132+133+134+135+136+137+138+139+140+141+142+143+144+145+146+147+148+149+150+151+152+153+154+155+156+157+158+159+160+161+162+163+164+165+166+167+168+169+170+171+172+173+174+175+176+177+178+179+180+181+182+183+184+185+186+187+188+189+190+191+192+193+194+195+196+197+198+199+200+201+202+203+204+205+206+207+208+209+210+211+212+213+214+215+216+217+218+219+220+221+222+223+224+225+226+227+228+229+230+231+232+233+234+235+236+237+238+239+240+241+242+243+244+245+246+247+248+249+250+251+252+253+254+255+256+257+258+259+260+261+262+263+264+265+266+267+268+269+270+271+272+273+274+275+276+277+278+279+280+281+282+283+284+285+286+287+288+289+290+291+292+293+294+295+296+297+298+299+300+301+302+303+304+305+306+307+308+309+310+311+312+313+314+315+316+317+318+319+320+321+322+323+324+325+326+327+328+329+330+331+332+333+334+335+336+337+338+339+340+341+342+343+344+345+346+347+348+349+350+351+352+353+354+355+356+357+358+359+360+361+362+363+364+365+366+367+368+369+370+371+372+373+374+375+376+377+378+379+380+381+382+383+384+385+386+387+388+389+390+391+392+393+394+395+396+397+398+399+400+401+402+403+404+405+406+407+408+409+410+411+412+413+414+415+416+417+418+419+420+421+422+423+424+425+426+427+428+429+430+431+432+433+434+435+436+437+438+439+440+441+442+443+444+445+446+447+448+449+450+451+452+453+454+455+456+457+458+459+460+461+462+463+464+465+466+467+468+469+470+471+472+473+474+475+476+477+478+479+480+481+482+483+484+485+486+487+488+489+490+491+492+493+494+495+496+497+498+499+500+501+502+503+504+505+506+507+508+509+510+511+512+513+514+515+516+517+518+519+520+521+522+523+524+525+526+527+528+529+530+531+532+533+534+535+536+537+538+539+540+541+542+543+544+545+546+547+548+549+550+551+552+553+554+555+556+557+558+559+560+561+562+563+564+565+566+567+568+569+570+571+572+573+574+575+576+577+578+579+580+581+582+583+584+585+586+587+588+589+590+591+592+593+594+595+596+597+598+599+600+601+602+603+604+605+606+607+608+609+610+611+612+613+614+615+616+617+618+619+620+621+622+623+624+625+626+627+628+629+630+631+632+633+634+635+636+637+638+639+640+641+642+643+644+645+646+647+648+649+650+651+652+653+654+655+656+657+658+659+660+661+662+663+664+665+666+667+668+669+670+671+672+673+674+675+676+677+678+679+680+681+682+683+684+685+686+687+688+689+690+691+692+693+694+695+696+697+698+699+700+701+702+703+704+705+706+707+708+709+710+711+712+713+714+715+716+717+718+719+720+721+722+723+724+725+726+727+728+729+730+731+732+733+734+735+736+737+738+739+740+741+742+743+744+745+746+747+748+749+750+751+752+753+754+755+756+757+758+759+760+761+762+763+764+765+766+767+768+769+770+771+772+773+774+775+776+777+778+779+780+781+782+783+784+785+786+787+788+789+790+791+792+793+794+795+796+797+798+799+800+801+802+803+804+805+806+807+808+809+810+811+812+813+814+815+816+817+818+819+820+821+822+823+824+825+826+827+828+829+830+831+832+833+834+835+836+837+838+839+840+841+842+843+844+845+846+847+848+849+850+851+852+853+854+855+856+857+858+859+860+861+862+863+864+865+866+867+868+869+870+871+872+873+874+875+876+877+878+879+880+881+882+883+884+885+886+887+888+889+890+891+892+893+894+895+896+897+898+899+900+901+902+903+904+905+906+907+908+909+910+911+912+913+914+915+916+917+918+919+920+921+922+923+924+925+926+927+928+929+930+931+932+933+934+935+936+937+938+939+940+941+942+943+944+945+946+947+948+949+950+951+952+953+954+955+956+957+958+959+960+961+962+963+964+965+966+967+968+969+970+971+972+973+974+975+976+977+978+979+980+981+982+983+984+985+986+987+988+989+990+991+992+993+994+995+996+997+998+999+1000+1001+1002+1003+1004+1005+1006+1007+1008+1009+1010+1011+1012+1013+1014+1015+1016+1017+1018+1019+1020+1021+1022+1023+1024+1025+1026+1027+1028+1029+1030+1031+1032+1033+1034+1035+1036+1037+1038+1039+1040+1041+1042+1043+1044+1045+1046+1047+1048+1049+1050+1051+1052+1053+1054+1055+1056+1057+1058+1059+1060+1061+1062+1063+1064+1065+1066+1067+1068+1069+1070+1071+1072+1073+1074+1075+1076+1077+1078+1079+1080+1081+1082+1083+1084+1085+1086+1087+1088+1089+1090+1091+1092+1093+1094+1095+1096+1097+1098+1099+1100+1101+1102+1103+1104+1105+1106+1107+1108+1109+1110+1111+1112+1113+1114+1115+1116+1117+1118+1119+1120+1121+1122+1123+1124+1125+1126+1127+1128+1129+1130+1131+1132+1133+1134+1135+1136+1137+1138+1139+1140+1141+1142+1143+1144+1145+1146+1147+1148+1149+1150+1151+1152+1153+1154+1155+1156+1157+1158+1159+1160+1161+1162+1163+1164+1165+1166+1167+1168+1169+1170+1171+1172+1173+1174+1175+1176+1177+1178+1179+1180+1181+1182+1183+1184+1185+1186+1187+1188+1189+1190+1191+1192+1193+1194+1195+1196+1197+1198+1199+1200+1201+1202+1203+1204+1205+1206+1207+1208+1209+1210+1211+1212+1213+1214+1215+1216+1217+1218+1219+1220+1221+1222+1223+1224+1225+1226+1227+1228+1229+1230+1231+1232+1233+1234+1235+1236+1237+1238+1239+1240+1241+1242+1243+1244+1245+1246+1247+1248+1249+1250+1251+1252+1253+1254+1255+1256+1257+1258+1259+1260+1261+1262+1263+1264+1265+1266+1267+1268+1269+1270+1271+1272+1273+1274+1275+1276+1277+1278+1279+1280+1281+1282+1283+1284+1285+1286+1287+1288+1289+1290+1291+1292+1293+1294+1295+1296+1297+1298+1299+1300+1301+1302+1303+1304+1305+1306+1307+1308+1309+1310+1311+1312+1313+1314+1315+1316+1317+1318+1319+1320+1321+1322+1323+1324+1325+1326+1327+1328+1329+1330+1331+1332+1333+1334+1335+1336+1337+1338+1339+1340+1341+1342+1343+1344+1345+1346+1347+1348+1349+1350+1351+1352+1353+1354+1355+1356+1357+1358+1359+1360+1361+1362+1363+1364+1365+1366+1367+1368+1369+1370+1371+1372+1373+1374+1375+1376+1377+1378+1379+1380+1381+1382+1383+1384+1385+1386+1387+1388+1389+1390+1391+1392+1393+1394+1395+1396+1397+1398+1399+1400+1401+1402+1403+1404+1405+1406+1407+1408+1409+1410+1411+1412+1413+1414+1415+1416+1417+1418+1419+1420+1421+1422+1423+1424+1425+1426+1427+1428+1429+1430+1431+1432+1433+1434+1435+1436+1437+1438+1439+1440+1441+1442+1443+1444+1445+1446+1447+1448+1449+1450+1451+1452+1453+1454+1455+1456+1457+1458+1459+1460+1461+1462+1463+1464+1465+1466+1467+1468+1469+1470+1471+1472+1473+1474+1475+1476+1477+1478+1479+1480+1481+1482+1483+1484+1485+1486+1487+1488+1489+1490+1491+1492+1493+1494+1495+1496+1497+1498+1499+1500+1501+1502+1503+1504+1505+1506+1507+1508+1509+1510+1511+1512+1513+1514+1515+1516+1517+1518+1519+1520+1521+1522+1523+1524+1525+1526+1527+1528+1529+1530+1531+1532+1533+1534+1535+1536+1537+1538+1539+1540+1541+1542+1543+1544+1545+1546+1547+1548+1549+1550+1551+1552+1553+1554+1555+1556+1557+1558+1559+1560+1561+1562+1563+1564+1565+1566+1567+1568+1569+1570+1571+1572+1573+1574+1575+1576+1577+1578+1579+1580+1581+1582+1583+1584+1585+1586+1587+1588+1589+1590+1591+1592+1593+1594+1595+1596+1597+1598+1599+1600+1601+1602+1603+1604+1605+1606+1607+1608+1609+1610+1611+1612+1613+1614+1615+1616+1617+1618+1619+1620+1621+1622+1623+1624+1625+1626+1627+1628+1629+1630+1631+1632+1633+1634+1635+1636+1637+1638+1639+1640+1641+1642+1643+1644+1645+1646+1647+1648+1649+1650+1651+1652+1653+1654+1655+1656+1657+1658+1659+1660+1661+1662+1663+1664+1665+1666+1667+1668+1669+1670+1671+1672+1673+1674+1675+1676+1677+1678+1679+1680+1681+1682+1683+1684+1685+1686+1687+1688+1689+1690+1691+1692+1693+1694+1695+1696+1697+1698+1699+1700+1701+1702+1703+1704+1705+1706+1707+1708+1709+1710+1711+1712+1713+1714+1715+1716+1717+1718+1719+1720+1721+1722+1723+1724+1725+1726+1727+1728+1729+1730+1731+1732+1733+1734+1735+1736+1737+1738+1739+1740+1741+1742+1743+1744+1745+1746+1747+1748+1749+1750+1751+1752+1753+1754+1755+1756+1757+1758+1759+1760+1761+1762+1763+1764+1765+1766+1767+1768+1769+1770+1771+1772+1773+1774+1775+1776+1777+1778+1779+1780+1781+1782+1783+1784+1785+1786+1787+1788+1789+1790+1791+1792+1793+1794+1795+1796+1797+1798+1799+1800+1801+1802+1803+1804+1805+1806+1807+1808+1809+1810+1811+1812+1813+1814+1815+1816+1817+1818+1819+1820+1821+1822+1823+1824+1825+1826+1827+1828+1829+1830+1831+1832+1833+1834+1835+1836+1837+1838+1839+1840+1841+1842+1843+1844+1845+1846+1847+1848+1849+1850+1851+1852+1853+1854+1855+1856+1857+1858+1859+1860+1861+1862+1863+1864+1865+1866+1867+1868+1869+1870+1871+1872+1873+1874+1875+1876+1877+1878+1879+1880+1881+1882+1883+1884+1885+1886+1887+1888+1889+1890+1891+1892+1893+1894+1895+1896+1897+1898+1899+1900+1901+1902+1903+1904+1905+1906+1907+1908+1909+1910+1911+1912+1913+1914+1915+1916+1917+1918+1919+1920+1921+1922+1923+1924+1925+1926+1927+1928+1929+1930+1931+1932+1933+1934+1935+1936+1937+1938+1939+1940+1941+1942+1943+1944+1945+1946+1947+1948+1949+1950+1951+1952+1953+1954+1955+1956+1957+1958+1959+1960+1961+1962+1963+1964+1965+1966+1967+1968+1969+1970+1971+1972+1973+1974+1975+1976+1977+1978+1979+1980+1981+1982+1983+1984+1985+1986+1987+1988+1989+1990+1991+1992+1993+1994+1995+1996+1997+1998+1999+2000+2001+2002+2003+2004+2005+2006+2007+2008+2009+2010+2011+2012+2013+2014+2015+2016+2017+2018+2019+2020+2021+2022+2023+2024+2025+2026+2027+2028+2029+2030+2031+2032+2033+2034+2035+2036+2037+2038+2039+2040+2041+2042+2043+2044+2045+2046+2047+2048+2049+2050+2051+2052+2053+2054+2055+2056+2057+2058+2059+2060+2061+2062+2063+2064+2065+2066+2067+2068+2069+2070+2071+2072+2073+2074+2075+2076+2077+2078+2079+2080+2081+2082+2083+2084+2085+2086+2087+2088+2089+2090+2091+2092+2093+2094+2095+2096+2097+2098+2099+2100+2101+2102+2103+2104+2105+2106+2107+2108+2109+2110+2111+2112+2113+2114+2115+2116+2117+2118+2119+2120+2121+2122+2123+2124+2125+2126+2127+2128+2129+2130+2131+2132+2133+2134+2135+2136+2137+2138+2139+2140+2141+2142+2143+2144+2145+2146+2147+2148+2149+2150+2151+2152+2153+2154+2155+2156+2157+2158+2159+2160+2161+2162+2163+2164+2165+2166+2167+2168+2169+2170+2171+2172+2173+2174+2175+2176+2177+2178+2179+2180+2181+2182+2183+2184+2185+2186+2187+2188+2189+2190+2191+2192+2193+2194+2195+2196+2197+2198+2199+2200+2201+2202+2203+2204+2205+2206+2207+2208+2209+2210+2211+2212+2213+2214+2215+2216+2217+2218+2219+2220+2221+2222+2223+2224+2225+2226+2227+2228+2229+2230+2231+2232+2233+2234+2235+2236+2237+2238+2239+2240+2241+2242+2243+2244+2245+2246+2247+2248+2249+2250+2251+2252+2253+2254+2255+2256+2257+2258+2259+2260+2261+2262+2263+2264+2265+2266+2267+2268+2269+2270+2271+2272+2273+2274+2275+2276+2277+2278+2279+2280+2281+2282+2283+2284+2285+2286+2287+2288+2289+2290+2291+2292+2293+2294+2295+2296+2297+2298+2299+2300+2301+2302+2303+2304+2305+2306+2307+2308+2309+2310+2311+2312+2313+2314+2315+2316+2317+2318+2319+2320+2321+2322+2323+2324+2325+2326+2327+2328+2329+2330+2331+2332+2333+2334+2335+2336+2337+2338+2339+2340+2341+2342+2343+2344+2345+2346+2347+2348+2349+2350+2351+2352+2353+2354+2355+2356+2357+2358+2359+2360+2361+2362+2363+2364+2365+2366+2367+2368+2369+2370+2371+2372+2373+2374+2375+2376+2377+2378+2379+2380+2381+2382+2383+2384+2385+2386+2387+2388+2389+2390+2391+2392+2393+2394+2395+2396+2397+2398+2399+2400+2401+2402+2403+2404+2405+2406+2407+2408+2409+2410+2411+2412+2413+2414+2415+2416+2417+2418+2419+2420+2421+2422+2423+2424+2425+2426+2427+2428+2429+2430+2431+2432+2433+2434+2435+2436+2437+2438+2439+2440+2441+2442+2443+2444+2445+2446+2447+2448+2449+2450+2451+2452+2453+2454+2455+2456+2457+2458+2459+2460+2461+2462+2463+2464+2465+2466+2467+2468+2469+2470+2471+2472+2473+2474+2475+2476+2477+2478+2479+2480+2481+2482+2483+2484+2485+2486+2487+2488+2489+2490+2491+2492+2493+2494+2495+2496+2497+2498+2499+2500+2501+2502+2503+2504+2505+2506+2507+2508+2509+2510+2511+2512+2513+2514+2515+2516+2517+2518+2519+2520+2521+2522+2523+2524+2525+2526+2527+2528+2529+2530+2531+2532+2533+2534+2535+2536+2537+2538+2539+2540+2541+2542+2543+2544+2545+2546+2547+2548+2549+2550+2551+2552+2553+2554+2555+2556+2557+2558+2559+2560+2561+2562+2563+2564+2565+2566+2567+2568+2569+2570+2571+2572+2573+2574+2575+2576+2577+2578+2579+2580+2581+2582+2583+2584+2585+2586+2587+2588+2589+2590+2591+2592+2593+2594+2595+2596+2597+2598+2599+2600+2601+2602+2603+2604+2605+2606+2607+2608+2609+2610+2611+2612+2613+2614+2615+2616+2617+2618+2619+2620+2621+2622+2623+2624+2625+2626+2627+2628+2629+2630+2631+2632+2633+2634+2635+2636+2637+2638+2639+2640+2641+2642+2643+2644+2645+2646+2647+2648+2649+2650+2651+2652+2

Figure 4: Query 2: Postgres Query Output

5.3.3 Observations

- Static resources such as images, icons, and logo files dominate the ranking because browsers repeatedly request these assets.
- NASA logo images and homepage-related resources appear among the most requested entries, indicating frequent access to common landing pages.
- Large distinct host counts show that the most popular resources were accessed by many unique clients rather than a small number of repeated users.

5.4 Query 3: Hourly Error Analysis

Query 3 computes the hourly error analysis by aggregating web server logs based on **log date** and **log hour**. For each combination, the system calculates:

- Total number of error requests (HTTP status codes 400–599)
- Total number of requests
- Error rate percentage
- Number of distinct hosts generating error requests

5.4.1 Implementation Details

The query is executed using a multi-stage approach:

- Each batch of logs is stored in a separate MongoDB collection.
- Aggregation is performed independently on each batch using MongoDB's **\$group** operator.
- Logs are grouped by **log_date** and **log_hour**.
- Error requests are identified using HTTP status codes in the range 400 to 599.
- Distinct hosts causing errors are collected using **\$addToSet**.
- Partial results from all batches are merged to compute global aggregates.
- Multi-threading is used to process batch collections in parallel, improving performance.
- The final results are sorted by log date and hour.

5.4.2 Sample Output

Figure 5 shows the console output generated after execution.

log_date	log_hour	error_request_count	total_request_count	error_rate	distinct_error_hosts
1995-07-01	0	24	3565	0.67	12
1995-07-01	1	10	3004	0.33	7
1995-07-01	2	12	2268	0.53	9
1995-07-01	3	16	1734	0.92	5
1995-07-01	4	10	1482	0.67	8
1995-07-01	5	9	1343	0.67	5
1995-07-01	6	4	1528	0.26	3
1995-07-01	7	2	1557	0.13	1
1995-07-01	8	6	1927	0.31	4
1995-07-01	9	3	2096	0.14	2
1995-07-01	10	13	3046	0.43	7
1995-07-01	11	23	3428	0.67	13
1995-07-01	12	10	2769	0.36	7
1995-07-01	13	13	3086	0.42	10
1995-07-01	14	19	3133	0.61	14
1995-07-01	15	17	3272	0.52	12
1995-07-01	16	14	3191	0.44	9

Figure 5: Query 3: Console Output

Figure 6 shows the Query 3 output stored in PostgreSQL.

	log_date text	log_hour integer	error_request_count integer	total_request_count integer	error_rate double precision	distinct_error_hosts integer	batch_id text	run_id text	pipeline_name text
1	1995-07-01	0	24	3565	0.67	12	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
2	1995-07-01	1	10	3004	0.33	7	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
3	1995-07-01	2	12	2268	0.53	9	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
4	1995-07-01	3	16	1734	0.92	5	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
5	1995-07-01	4	10	1482	0.67	8	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
6	1995-07-01	5	9	1343	0.67	5	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
7	1995-07-01	6	4	1528	0.26	3	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
8	1995-07-01	7	2	1557	0.13	1	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
9	1995-07-01	8	6	1927	0.31	4	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
10	1995-07-01	9	3	2096	0.14	2	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
11	1995-07-01	10	13	3046	0.43	7	1	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
12	1995-07-01	11	23	3428	0.67	13	1+2	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
13	1995-07-01	12	10	2769	0.36	7	2	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
14	1995-07-01	13	13	3086	0.42	10	2	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
15	1995-07-01	14	19	3133	0.61	14	2	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
16	1995-07-01	15	17	3272	0.52	12	2	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
17	1995-07-01	16	14	3191	0.44	9	2	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
18	1995-07-01	17	15	3671	0.41	9	2	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
19	1995-07-01	18	15	3192	0.47	11	2	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb
20	1995-07-01	19	16	2831	0.57	11	2+3	0ea6b66d-e469-4b06-a9ea-f52a0c6b4f2c	mongodb

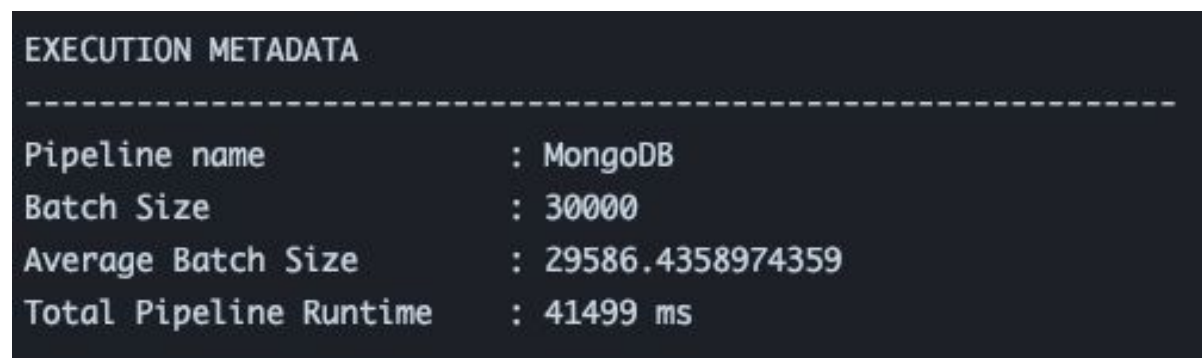
Figure 6: Query 3: PostgreSQL Output

5.4.3 Observations

- Hours with higher error counts indicate peak failure periods in the system.
- The error rate highlights time intervals where failures form a significant portion of total traffic.
- Distinct error hosts help identify whether failures originate from many clients or a limited set of sources.

- Batch identifiers confirm that multiple batches contribute to the final aggregated results.
- The aggregation correctly combines results across all batches, validating the global merge logic.

6. Runtime Module



```
EXECUTION METADATA
-----
Pipeline name      : MongoDB
Batch Size         : 30000
Average Batch Size : 29586.4358974359
Total Pipeline Runtime : 41499 ms
```

EXECUTION METADATA	

Pipeline name	: MongoDB
Batch Size	: 30000
Average Batch Size	: 29586.4358974359
Total Pipeline Runtime	: 41499 ms

Figure 7: Runtime Module output

Conclusion

In this phase, we implemented the MongoDB-based pipeline of the ETL framework for processing large-scale web server logs. The system successfully performs log parsing, batch-wise data processing, and aggregation using MongoDB’s pipeline framework.

Query execution is optimized through multi-threaded processing of batch collections, and partial results are merged to produce correct global aggregates. The final results are enriched with metadata and stored directly in PostgreSQL for persistent reporting.

Overall, the MongoDB pipeline demonstrates efficient handling of large datasets, scalability through batching, and accurate aggregation across multiple batches.